Important C++ Program Questions - PSOOP Assignment-II

Inheritance and Access Specifiers:

1. Create a base class `Animal` with a protected member variable `age`. Derive a class `Dog` from `Animal` and write a member function in `Dog` to set and display the age of the dog. Demonstrate the access of `age` directly from an object of `Dog` outside the class. Explain the use of protected access specifiers in inheritance.

2. Imagine a publishing company that markets both book and audiocassette versions of its works. Create a class `publication` that stores the title (a string) and price (float). Derive two classes: `book`, which adds a page count (int), and `tape`, which adds a playing time in minutes (float). Each class should have `getdata()` and `putdata()` functions. Write a `main()` to test the classes.

3. Write a program showing the order of constructor and destructor calls in the base and derived classes.

4. Write a C++ program to demonstrate the diamond problem and solve it using virtual base class. Construct a base class `Person` inherited by `Teacher` and `Researcher`, and a derived class `Professor` that inherits both.

5. Can a derived class access private members of the base class? Write a C++ program to show that a derived class cannot directly access private members of the base class.

Multiple Inheritance:

1. Write a program to declare classes `X`, `Y`, and `Z`. Each class contains a `char` array as a data member. Apply multiple inheritance. Concatenate all three `char` arrays. Show the resultant char array using constructor and destructor.

2. Construct a derived class `Car` with member function `drive()` which inherits from classes `Engine` (with `startEngine()`) and `GPS` (with `startGPS()`) to showcase multiple inheritance.

Operator Overloading:

1. Write a C++ program that takes two values of time (hr, min, sec) and outputs their sum using

constructor and operator overloading.

2. It is desired to add and multiply two complex numbers using operator overloading. Write a C++ program to accomplish the same. Also, discuss the overloaded `+` operator used in the program.

3. Write a class `DATE`, use overloaded `+` operator to add two dates and display the result. Assume non-leap year dates.

4. Create a class `Distance`, with members meter and cm, and create an overloaded `*` operator using a friend function so you can use expressions like `Wdist1 = 7.5 * dist2;`. Include a one-argument constructor to convert float to Distance.

5. Write a program to display compile-time polymorphism and overload the `++` operator for a class that simulates a dynamic array.

6. Write a program to overload the relational operator `==` to compare two objects of a class `Box` based on dimensions (length, width, height). Use a parameterized constructor.

Polymorphism:

1. Write a C++ program using runtime polymorphism to simulate a zoo, where animals like Lion, Tiger, and Elephant inherit from a base class `Animal` and override a `makeSound()` function. Create a base class pointer to point to derived class objects.

2. Write a C++ program to implement a simple banking system using polymorphism. Each account type (`SavingsAccount`, `CheckingAccount`) overrides a `calculateInterest()` method from a common `Account` base class.

3. Design a system that simulates the behavior of various types of vehicles (e.g., `Car`, `Motorcycle`, `Bicycle`) using polymorphism. Each should have a `move()` method overridden in derived classes.

4. Write a C++ program that demonstrates runtime polymorphism with an abstract class `Shape` that declares a pure virtual function `draw()`. The classes `Rectangle` and `Circle` implement `draw()`. In `main()`, use a base class pointer to call the `draw()` function on derived objects.

Exception Handling:

1. Write a C++ program to compute the square root of a number. If the number is negative, raise an exception.

2. Write a program that reads two numbers, `u` and `v`. It computes `u/v`. Function checks for `v`, and if it is 0, then return exception.

3. Write a C++ program using a custom exception class to handle invalid age input (e.g., age < 0). Create a `Person` class with `name` and `age`. Implement `getAge()` and throw exception `incorrectAge` if age < 0. Handle it in `main()`.

4. Create a class `Student` with rollno, name and marks of three subjects. Use a parameterized constructor to input details and throw exception if marks (<=0 or >100). Create `invalidMarks` exception class and throw a temporary object.

5. Create a class `BankAccount` that throws an exception when a withdrawal exceeds the balance. Use the exception class `NegativeBalance` from `withdraw()`.

6. Create a `BankAccount` class with `accno` and `balance`. Use a parametrized constructor, and functions `deposit()` and `withdraw()`. Throw an exception when withdrawal exceeds balance. Provide a catch-all handler for all exceptions.

File Handling:

1. Write a C++ program to read the content of a text file and display it on the console.

2. Write a C++ program that reads a file and counts the number of sentences, words, and characters.

3. Write a C++ program to copy the contents of one file to a new file by removing unnecessary spaces between words.

4. Create a class `Student` with data members `name`, `age`, `gender`, `height`, and `weight`. Use member functions to get data from keyboard. Store data in a file using `read()` and `write()` methods.

5. Write a C++ program to read 10 odd numbers. Generate the exception for negative and even numbers. Provide a handler to display an error message.

6. Write a program to count the frequency of each character in a text file including special characters, digits, and whitespace. Handle file not found errors.

7. Create a C++ program to append data to an existing file. Handle the case when the file does not exist or fails to open.

8. Write a program to read an existing file, check if each character is a vowel, and display it. Handle file open failure.